



Eastern Mediterranean University

School of Computing and Technology
Cloud Computing Applications

Prepared by

Abdulrahman Ahmed	22902545
Osman Eid	23704362
Mohamed Allafi	22901510
Abdulmalik Khalid	22901037
Ali Mohammed	23704831

Section One: Project Objective

The objective of this project is to design, build, and test a simple RESTful Web Service using ASP.NET Core Web API. The application chosen for this assignment is a Clinic Appointment Management API, named ClinicApi, which allows a client to manage patient appointments through a set of HTTP endpoints.

The main goal of the project is to understand how a Web API is structured internally — from the data model, to the business logic layer, to the controller that exposes the endpoints — and to learn how to test and verify that a REST API behaves correctly using an API testing tool such as Postman. The project demonstrates the full lifecycle of a resource (an appointment) through Create, Read, Update, and Delete (CRUD) operations, each mapped to the appropriate HTTP method (POST, GET, PUT, and DELETE).

By completing this project, the team gained practical experience with REST API design principles, JSON-based request and response handling, and the use of Postman to validate that each endpoint returns the expected status code and payload.

Section Two: Technologies Used

The following technologies and tools were used to build, run, and test the Web Service:

- ASP.NET Core Web API (C#) – used to build the RESTful Web Service.
- Visual Studio / Visual Studio Code – development environment used to write and debug the project.
- .NET SDK – used to compile and run the API on a local Kestrel web server (<http://localhost:5109>).
- JSON (JavaScript Object Notation) – the data format used for all request and response bodies exchanged with the API.
- Postman – used as the API Testing tool to send HTTP requests (GET, POST, PUT, DELETE) to the API's endpoints and inspect the responses.
- Dependency Injection (built into ASP.NET Core) – used to register and inject the AppointmentService into the controller through the IAppointmentService interface.

Although the API in this project was implemented as a REST API rather than a SOAP API, both approaches were researched as part of the assignment. REST relies on standard HTTP Methods and lightweight JSON payloads, while a SOAP API uses XML-based messages and a formal Service Contract described by a WSDL (Web Services Description Language) document. REST was selected for this project because it is simpler to implement, easier to test with tools such as Postman, and is the most common style used for modern Web Services.

Section Three: Model Class Created

The data used by the Web Service is represented by the Appointment model class, located in the ClinicApi.Models namespace. This class defines the structure of the resource that the API exposes and that is serialized to and from JSON in every request and response.

The Appointment class contains the following properties:

- Id (int) – the unique identifier of the appointment, generated automatically by the service.
- PatientName (string) – the name of the patient booking the appointment.
- DoctorName (string) – the name of the doctor the appointment is scheduled with.
- AppointmentDate (DateTime) – the date and time of the appointment.
- Reason (string) – the reason for the visit (e.g., "General check-up").
- Status (string) – the current status of the appointment, defaulting to "Scheduled".

```

namespace ClinicApi.Models;

17 references
public class Appointment
{
    5 references
    public int Id { get; set; }

    3 references
    public string PatientName { get; set; } = string.Empty;

    3 references
    public string DoctorName { get; set; } = string.Empty;

    3 references
    public DateTime AppointmentDate { get; set; }

    3 references
    public string Reason { get; set; } = string.Empty;

    3 references
    public string Status { get; set; } = "Scheduled";
}

```

Figure 3.1 – The Appointment model class (ClinicApi.Models)

Model code (Appointment.cs)

```

namespace ClinicApi.Models;

public class Appointment
{
    public int Id { get; set; }
    public string PatientName { get; set; } = string.Empty;
    public string DoctorName { get; set; } = string.Empty;
    public DateTime AppointmentDate { get; set; }
    public string Reason { get; set; } = string.Empty;
    public string Status { get; set; } = "Scheduled";
}

```

Section Four: Service Class and Business Logic

All of the business logic for managing appointments is contained in a dedicated service layer rather than in the controller. This keeps the controller thin and makes the logic reusable and easier to test. The contract for this layer is defined by the

IAppointmentService interface, which declares the five core operations required by the assignment: listing, viewing details, adding, updating, and deleting.

```
using ClinicApi.Models;

namespace ClinicApi.Services;

4 references
public interface IAppointmentService
{
    2 references
    List<Appointment> GetAll();

    4 references
    Appointment? GetById(int id);

    2 references
    Appointment Add(Appointment appointment);

    2 references
    bool Update(int id, Appointment appointment);

    2 references
    bool Delete(int id);
}
```

Figure 4.1 – The IAppointmentService interface

```
public interface IAppointmentService
{
    List<Appointment> GetAll();
    Appointment? GetById(int id);
    Appointment Add(Appointment appointment);
    bool Update(int id, Appointment appointment);
    bool Delete(int id);
}
```

Retrieving Data – GetAll and GetById

The GetAll() method returns the full in-memory list of appointments, and is used to serve the GET /api/appointments endpoint. The GetById(int id) method searches the list using LINQ's FirstOrDefault and returns either the matching Appointment

or null if no appointment with that id exists — this null result is what allows the controller to return a 404 Not Found response.

```
2 references
public List<Appointment> GetAll()
{
    return _appointments;
}

4 references
public Appointment? GetById(int id)
{
    return _appointments.FirstOrDefault(a => a.Id == id);
}
```

Figure 4.2 – GetAll() and GetById() methods

```
public List<Appointment> GetAll()
{
    return _appointments;
}

public Appointment? GetById(int id)
{
    return _appointments.FirstOrDefault(a => a.Id == id);
}
```

Adding Data – Add

The Add(Appointment appointment) method contains the logic for generating a new, unique Id for every appointment that is created. If the list is empty, the first appointment is assigned Id = 1; otherwise, the new Id is calculated as the current maximum Id in the list plus one. The new appointment is then added to the in-memory collection and returned to the caller, which allows the controller to send it back to the client together with a 201 Created response.

```

2 references
public Appointment Add(Appointment appointment)
{
    appointment.Id = _appointments.Count == 0
        ? 1
        : _appointments.Max(a => a.Id) + 1;

    _appointments.Add(appointment);
    return appointment;
}

```

Figure 4.3 – Add() method

```

public Appointment Add(Appointment appointment)
{
    appointment.Id = _appointments.Count == 0
        ? 1
        : _appointments.Max(a => a.Id) + 1;

    _appointments.Add(appointment);
    return appointment;
}

```

Updating Data – Update

The Update(int id, Appointment appointment) method first retrieves the existing appointment using GetById(). If it is not found, the method returns false so the controller can respond appropriately. If it exists, each field (PatientName, DoctorName, AppointmentDate, Reason, and Status) is overwritten with the new values supplied in the request body, and the method returns true to signal success.

```

public bool Update(int id, Appointment appointment)
{
    var existingAppointment = GetById(id);

    if (existingAppointment is null)
    {
        return false;
    }

    existingAppointment.PatientName = appointment.PatientName;
    existingAppointment.DoctorName = appointment.DoctorName;
    existingAppointment.AppointmentDate = appointment.AppointmentDate;
    existingAppointment.Reason = appointment.Reason;
    existingAppointment.Status = appointment.Status;

    return true;
}

```

Figure 4.4 – Update() method

```

public bool Update(int id, Appointment appointment)
{
    var existingAppointment = GetById(id);
    if (existingAppointment is null)
    {
        return false;
    }

    existingAppointment.PatientName = appointment.PatientName;
    existingAppointment.DoctorName = appointment.DoctorName;
    existingAppointment.AppointmentDate = appointment.AppointmentDate;
    existingAppointment.Reason = appointment.Reason;
    existingAppointment.Status = appointment.Status;

    return true;
}

```

Deleting Data – Delete

The Delete(int id) method also relies on GetById() to locate the appointment. If the appointment does not exist, it returns false; otherwise the appointment is removed from the in-memory list and the method returns true. This result is used by the controller to decide between a 204 No Content and a 404 Not Found response.

```

2 references
public bool Delete(int id)
{
    var appointment = GetById(id);

    if (appointment is null)
    {
        return false;
    }

    _appointments.Remove(appointment);
    return true;
}

```

Figure 4.5 – Delete() method

```

public bool Delete(int id)
{
    var appointment = GetById(id);
    if (appointment is null)
    {
        return false;
    }

    _appointments.Remove(appointment);
    return true;
}

```

Section Five: Controller and endpoint descriptions

The AppointmentsController exposes the business logic implemented in AppointmentService as a set of HTTP endpoints under the base route `api/appointments`. The controller depends on `IAppointmentService` (injected through the constructor via dependency injection) rather than depending on the concrete service class directly, which follows good REST API design practice and keeps the controller decoupled from the implementation details.

The endpoints exposed by the controller, confirmed through testing in Postman, are:

- GET `/api/appointments` – returns the full list of appointments.

- GET /api/appointments/{id} – returns a single appointment by its id, or a 404 Not Found message ("Appointment not found.") if the id does not exist.
- POST /api/appointments – creates a new appointment from the JSON request body and returns the created resource with a 201 Created status.
- PUT /api/appointments/{id} – updates an existing appointment with the JSON request body and returns 204 No Content on success.
- DELETE /api/appointments/{id} – deletes the appointment with the given id and returns 204 No Content on success.

Each action method in the controller calls the corresponding method on `IAppointmentService` and translates the result into the correct HTTP status code — for example, returning `NotFound()` when the service returns null or false, `Ok()` when data is successfully retrieved, `CreatedAtAction()` after a successful POST, and `NoContent()` after a successful PUT or DELETE.

```
namespace ClinicApi.Controllers;

[ApiController]
[Route("api/[controller]")]
1 reference
public class AppointmentsController : ControllerBase
{
    6 references
    private readonly IAppointmentService _appointmentService;

    0 references
    public AppointmentsController(IAAppointmentService appointmentService)
    {
        _appointmentService = appointmentService;
    }

    [HttpGet]
    0 references
    public ActionResult<List<Appointment>> GetAll()
    {
        return Ok(_appointmentService.GetAll());
    }

    [HttpGet("{id}")]
    1 reference
    public ActionResult<Appointment> GetById(int id)
    {
        var appointment = _appointmentService.GetById(id);

        if (appointment is null)
        {
            return NotFound(new { message = "Appointment not found." });
        }

        return Ok(appointment);
    }
}
```

Figure 5.1 – *AppointmentsController*: constructor, *GetAll*, and *GetById* actions

```

[HttpPost]
0 references
public ActionResult<Appointment> Add(Appointment appointment)
{
    var createdAppointment = _appointmentService.Add(appointment);

    return CreatedAtAction(
        nameof(GetById),
        new { id = createdAppointment.Id },
        createdAppointment);
}

[HttpPut("{id}")]
0 references
public IActionResult Update(int id, Appointment appointment)
{
    var updated = _appointmentService.Update(id, appointment);

    if (!updated)
    {
        return NotFound(new { message = "Appointment not found." });
    }

    return NoContent();
}

[HttpDelete("{id}")]
0 references
public IActionResult Delete(int id)
{
    var deleted = _appointmentService.Delete(id);

    if (!deleted)
    {
        return NotFound(new { message = "Appointment not found." });
    }

    return NoContent();
}

```

Figure 5.2 – AppointmentsController: Add, Update, and Delete actions

Controller code (AppointmentsController.cs)

```

namespace ClinicApi.Controllers;

[ApiController]
[Route("api/[controller]")]
public class AppointmentsController : ControllerBase
{
    private readonly IAppointmentService _appointmentService;

    public AppointmentsController(IAppointmentService appointmentService)

```

```

{
    _appointmentService = appointmentService;
}

[HttpGet]
public ActionResult<List<Appointment>> GetAll()
{
    return Ok(_appointmentService.GetAll());
}

[HttpGet("{id}")]
public ActionResult<Appointment> GetById(int id)
{
    var appointment = _appointmentService.GetById(id);
    if (appointment is null)
        return NotFound(new { message = "Appointment not found." });
    return Ok(appointment);
}

[HttpPost]
public ActionResult<Appointment> Add(Appointment appointment)
{
    var createdAppointment = _appointmentService.Add(appointment);
    return CreatedAtAction(nameof(GetById),
        new { id = createdAppointment.Id }, createdAppointment);
}

[HttpPut("{id}")]
public IActionResult Update(int id, Appointment appointment)
{
    var updated = _appointmentService.Update(id, appointment);
    if (!updated)
        return NotFound(new { message = "Appointment not found." });
    return NoContent();
}

[HttpDelete("{id}")]
public IActionResult Delete(int id)
{
    var deleted = _appointmentService.Delete(id);
    if (!deleted)
        return NotFound(new { message = "Appointment not found." });
    return NoContent();
}
}

```

Section Six: HTTP methods Used

The Web API follows REST conventions by using the standard HTTP Methods to represent the different operations that can be performed on the appointment

resource. The table below summarizes each HTTP method used, the endpoint it is mapped to, and the response that the API returns.

HTTP Method	Endpoint	Purpose	Typical Response
GET	/api/appointments	Retrieve the list of all appointments	200 OK + JSON array
GET	/api/appointments/{id}	Retrieve one appointment by id	200 OK, or 404 Not Found
POST	/api/appointments	Create a new appointment	201 Created + JSON object
PUT	/api/appointments/{id}	Update an existing appointment	204 No Content, or 404 Not Found
DELETE	/api/appointments/{id}	Delete an appointment	204 No Content, or 404 Not Found

Using distinct HTTP verbs for distinct operations (rather than a single generic endpoint) is one of the defining characteristics of REST API design, and it made the API predictable and straightforward to test in Postman.

Section Seven: Postman or SoapUI test screenshots

Because the API was built as a REST service, Postman was used as the API Testing tool to test every endpoint (rather than SoapUI, which is typically used for testing SOAP APIs based on a WSDL/Service Contract). Each request below was sent to the API running locally at <http://localhost:5109>, and the response body, status code, and timing were verified for each one.

GET – Retrieve all appointments

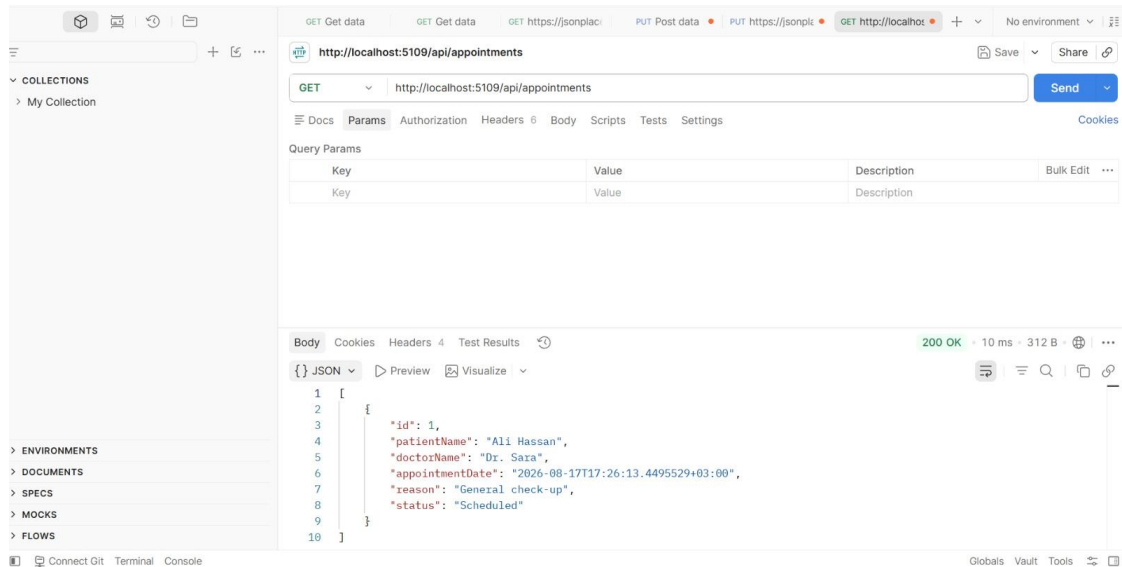


Figure 7.1 – GET /api/appointments returning 200 OK with a JSON array of appointments

GET – Retrieve a single appointment by id

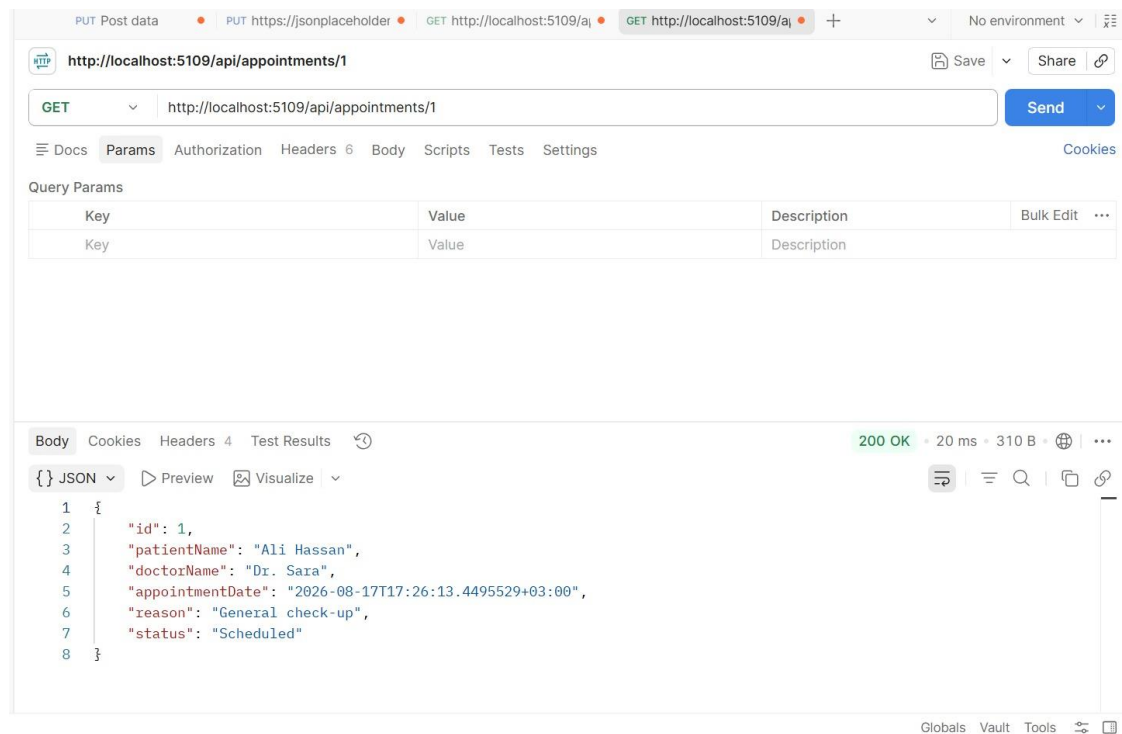


Figure 7.2 – GET /api/appointments/1 returning 200 OK with a single JSON object

POST – Create a new appointment

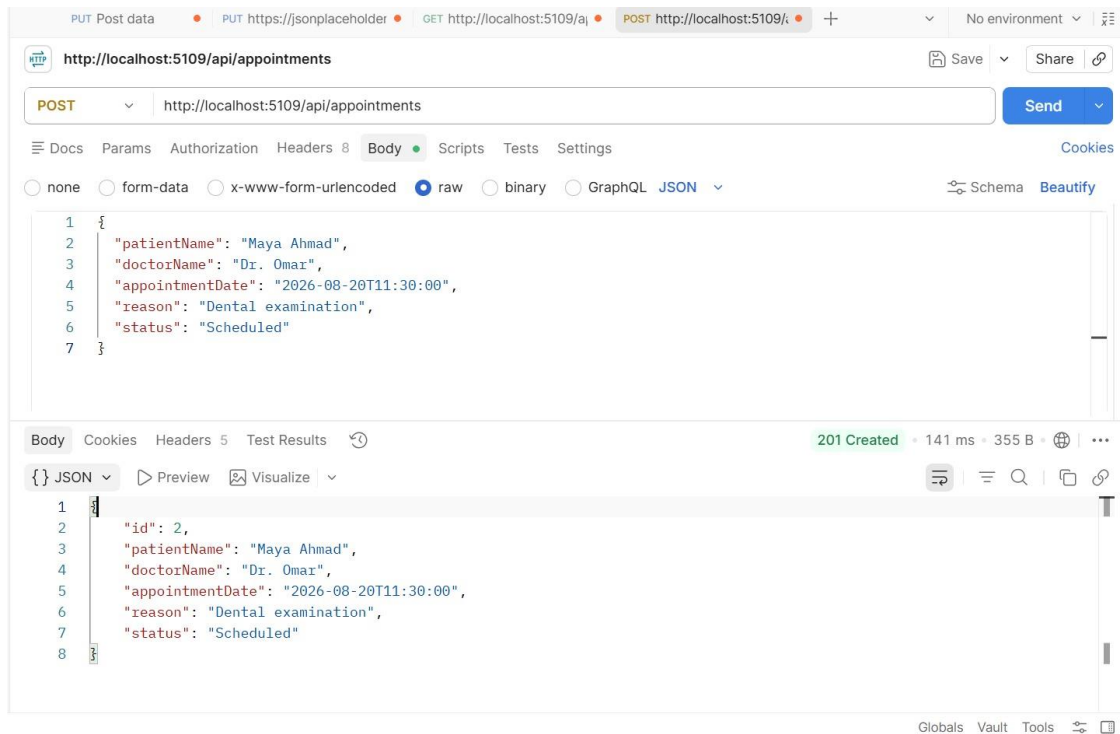


Figure 7.3 – POST /api/appointments returning 201 Created

PUT – Update an existing appointment

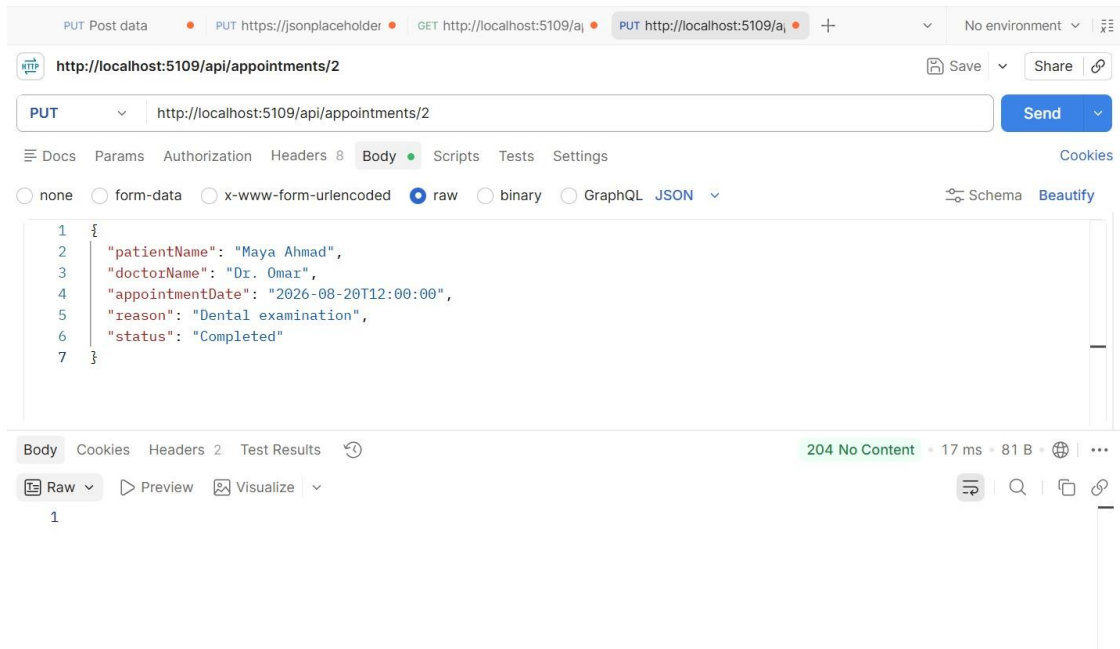


Figure 7.4 – PUT /api/appointments/2 returning 204 No Content

DELETE – Remove an appointment

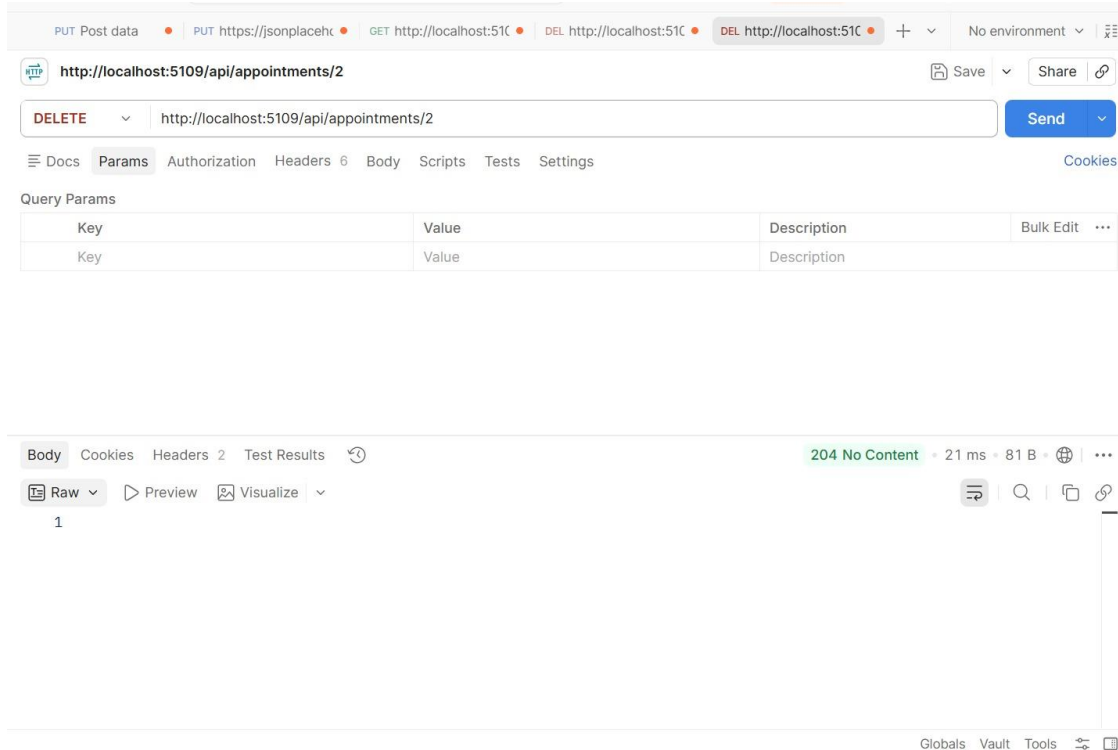


Figure 7.5 – DELETE /api/appointments/2 returning 204 No Content

Section Eight: Request/Response examples

This section lists the raw JSON Request and Response bodies exchanged with the API during testing, corresponding to the Postman screenshots in Section Seven.

1. GET /api/appointments/1 — Response (200 OK)

```
{
  "id": 1,
  "patientName": "Ali Hassan",
  "doctorName": "Dr. Sara",
  "appointmentDate": "2026-08-17T17:26:13.4495529+03:00",
  "reason": "General check-up",
  "status": "Scheduled"
}
```

2. POST /api/appointments — Request Body

```
{
  "patientName": "Maya Ahmad",
  "doctorName": "Dr. Omar",
  "appointmentDate": "2026-08-20T11:30:00",
}
```

```
{
  "reason": "Dental examination",
  "status": "Scheduled"
}
```

2. POST /api/appointments — Response (201 Created)

```
{
  "id": 2,
  "patientName": "Maya Ahmad",
  "doctorName": "Dr. Omar",
  "appointmentDate": "2026-08-20T11:30:00",
  "reason": "Dental examination",
  "status": "Scheduled"
}
```

3. PUT /api/appointments/2 — Request Body

```
{
  "patientName": "Maya Ahmad",
  "doctorName": "Dr. Omar",
  "appointmentDate": "2026-08-20T12:00:00",
  "reason": "Dental examination",
  "status": "Completed"
}
```

Response: 204 No Content — the update was applied successfully and no response body is returned.

4. DELETE /api/appointments/2 — Response

204 No Content — the appointment was found and removed successfully.

5. Error handling example — GET /api/appointments/99

When a client requests an appointment id that does not exist, the API does not return an empty 200 OK response. Instead, GetById() returns null, and the controller translates this into a 404 Not Found status together with a JSON error message, as shown below.

```
{
```

```
"message": "Appointment not found."
}
```

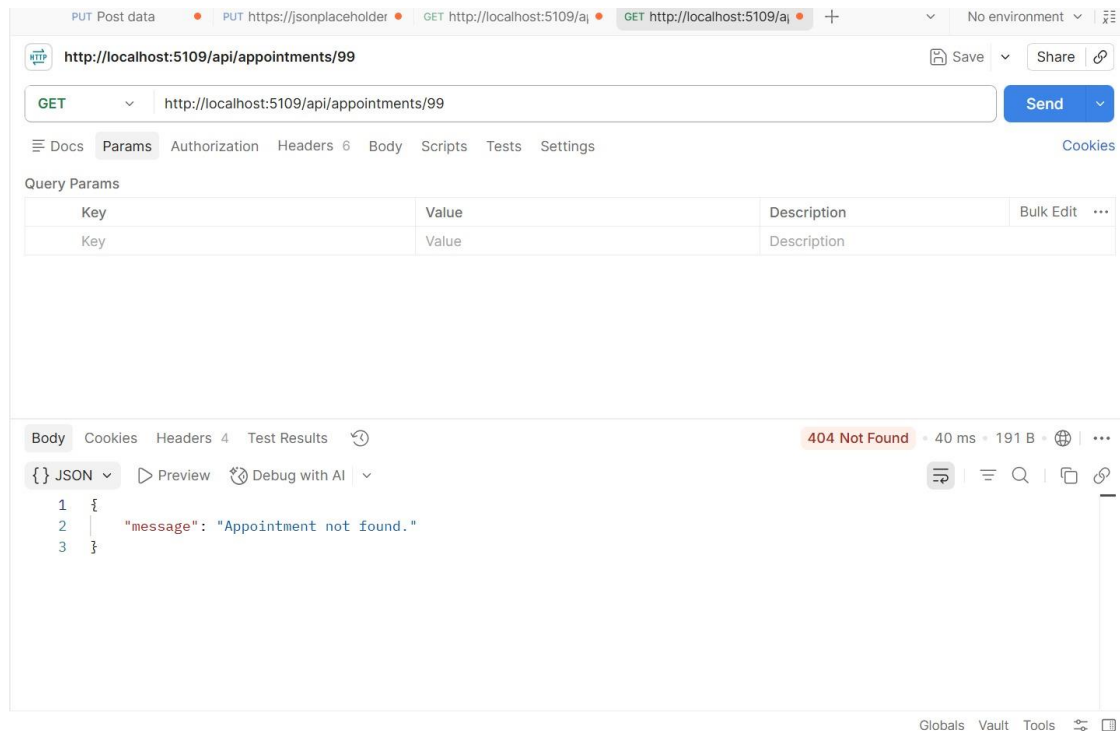


Figure 8.1 – GET /api/appointments/99 returning 404 Not Found

Section Nine: Conclusion and Evaluation

This project provided hands-on experience with the complete process of building and testing a REST API using ASP.NET Core. Starting from a simple Appointment model, the team implemented a service layer to hold the business logic, a controller to expose that logic as HTTP endpoints, and used Postman to verify that every endpoint behaved as expected.

Working through this assignment reinforced the difference between a Request and its Response, the meaning of each HTTP status code returned by the API (200, 201, 204, and 404), and how JSON is used as the common data format between the client and the Web Service. It also clarified how REST differs from SOAP: REST uses simple HTTP Methods and JSON over lightweight Endpoint URLs, while SOAP relies on a stricter Service Contract, XML messages, and a WSDL definition — which explains why REST APIs are generally faster to build and test with a tool such as Postman.

Overall, the API met all of the assignment's requirements: it supports full CRUD functionality, it does not use hardcoded data (all data lives in and is returned from the service layer), and every endpoint was verified independently in Postman before being documented in this report. A possible improvement for the future would be replacing the in-memory list with a real database (e.g., using Entity Framework Core) so that data persists between application restarts, and adding input validation to reject malformed requests before they reach the service layer.